OS-Hackthon Readme

READERS–WRITERS DATABASE SIMULATOR

Student Marks Management Using Shared Memory

1. Project Overview

The Readers–Writers Database Simulator is a mini-project developed using the C programming language. It demonstrates how multiple programs can access and view the same shared data.

In this project, student marks are used as an example. The system consists of one writer program and two reader programs running in three separate Ubuntu terminals. The writer enters student roll numbers, subjects, and marks. The reader programs display the student records and automatically detect changes made by the writer.

For example, if the writer enters a student's English marks as 92 and later updates them to 98, both readers display the previous marks and the updated marks.

This project helps us understand shared memory, synchronization, concurrency, and the Readers–Writers problem in operating systems.

2. Problem Statement

When multiple programs access the same data, it is important to coordinate their operations. If one program updates a record while another program is reading it, the data must be protected to avoid inconsistent results.

The aim of this project is to develop a simple student marks simulator in which one writer can add or update student marks while multiple readers view the latest records. Synchronization is used to protect the shared data during reading and writing operations.

3. Objectives

1. To understand the Readers–Writers problem in operating systems.
2. To implement a shared student marks database using C.
3. To allow the writer to enter a student's roll number, subject, and marks.
4. To allow existing marks to be updated after confirmation.
5. To display previous marks and current marks.
6. To run two reader programs in separate terminals.
7. To update the reader displays when the writer changes a record.

8. To understand shared memory and mutex synchronization.
9. To gain practical experience with C programming and Ubuntu Linux.

4. Technologies Used

C Programming Language: Used to develop the writer and reader programs.

Ubuntu Linux: The operating system used to run the project.

GCC Compiler: Used to compile the C source files into executable programs.

POSIX Shared Memory: Allows separate programs to access the same data stored in shared memory.

POSIX Threads (pthreads): Provides mutex functionality used to coordinate access to shared data.

Mutex Synchronization: Protects the shared records while the writer updates them and while readers copy them for display.

Linux Terminal: Used to compile, execute, and test the programs.

5. Project File Structure

The project contains three main files:

student_marks/

- shared.h
- writer.c
- reader.c

shared.h: Defines the shared-memory name, student structure, maximum number of records, and shared database structure. Both programs use this header file.

writer.c: Creates the shared-memory area, accepts student details, adds new records, updates existing marks, and records the previous marks before an update.

reader.c: Opens the shared-memory area, reads the student records safely, checks whether the data has changed, and displays the latest records.

6. Working Process

The project works through the following steps:

1. The writer program is started in the first terminal.
2. The writer creates the shared-memory area and initializes the mutex.
3. Reader 1 is started in the second terminal.
4. Reader 2 is started in the third terminal.

5. The writer enters the student's roll number and subject.
6. If the record is new, the writer enters marks and adds the record.
7. If the same roll number and subject already exist, the writer displays the current marks.
8. The writer asks whether the marks should be updated.
9. If the user enters “y”, the writer accepts the new marks and stores the previous marks.
10. If the user enters “n”, the existing marks remain unchanged.
11. The writer updates the shared data and increases a version number.
12. Both readers check the version number periodically.
13. When a change is detected, the readers display the updated student records.

7. Implementation Details

7.1 Writer Program

The writer program is responsible for adding and updating student records.

For a new record, the writer asks for:

- Roll number
- Subject
- Marks between 0 and 100

If the same roll number and subject already exist, the program displays the current marks and asks:

“Do you want to update marks? (y/n):”

Entering “y” allows the user to enter new marks. Entering “n” cancels the update.

The program stores the previous marks in the old_marks field and the latest marks in the marks field.

Entering roll number 0 stops the writer program.

7.2 Reader Programs

The reader program displays the student records stored in shared memory.

Two instances of the same reader executable are started in separate terminals. Both readers access the same shared data and periodically check for changes.

When the writer adds or updates a record, the readers detect the change and print the latest table.

The readers do not require separate copies of the database because both use the same shared-memory area.

7.3 Synchronization

A process-shared mutex is used to protect the shared data.

The writer locks the mutex while searching for or changing a record. A reader locks the mutex while copying the records and version number.

This prevents a reader from copying the shared records while the writer is in the middle of modifying them.

8. Compilation and Execution

All three files must be saved inside the student_marks directory.

To enter the project directory, run:

cd ~/student_marks

Compile the writer program:

gcc writer.c -o writer -pthread -lrt

Compile the reader program:

gcc reader.c -o reader -pthread -lrt

If compilation completes without errors, the executable programs are ready.

Terminal 1 — Writer

Open the first terminal and enter:

cd ~/student_marks

./writer

Terminal 2 — Reader 1

Open the second terminal and enter:

cd ~/student_marks

./reader

Terminal 3 — Reader 2

Open the third terminal and enter:

cd ~/student_marks

./reader

The writer should be started first because it creates the shared-memory area required by the readers.

9. Sample Output and Demonstration

9.1 Adding a New Student Record

In the writer terminal:

Enter roll number: 103

Enter subject: English

Enter marks (0-100): 92

New student subject record added.

Shared database updated. Readers will see the change.

Both reader terminals display the new record:

Roll No. | Subject | Old Marks | New Marks

103 | English | 92 | 92

9.2 Updating Existing Marks

The writer enters the same roll number and subject again:

Enter roll number: 103

Enter subject: English

Current marks: 92

Do you want to update marks? (y/n): y

Enter new marks (0-100): 98

Marks updated: 92 -> 98

Shared database updated. Readers will see the change.

Both reader terminals then display:

Roll No. | Subject | Old Marks | New Marks

103 | English | 92 | 98

This demonstrates that both reader programs can detect and display changes made by the writer.

9.3 Cancelling an Update

If the writer enters “n” when asked whether to update marks, the program keeps the existing marks unchanged.

9.4 Adding Another Subject

A student can have separate records for different subjects. For example, roll number 103 can have one English record and another Mathematics record.

A record is identified by the combination of roll number and subject.

10. How the Readers–Writers Concept Is Demonstrated

The Readers–Writers problem deals with coordinating access to shared data when reading and writing operations may happen at the same time.

In this project, the writer adds or updates student records, while two reader processes display the shared records. The mutex protects the data during changes and copying operations.

The project demonstrates shared access and synchronization. It uses a mutex around the shared data structure rather than a dedicated reader-writer lock that allows multiple readers to access that structure concurrently.

11. Advantages

1. Simple and easy to understand.
2. Demonstrates operating-system synchronization concepts through a real-world example.
3. Allows student marks to be added and updated.
4. Shows previous and current marks.
5. Allows two reader programs to display the same shared data.
6. Provides practical experience with shared memory in C.
7. Can be extended with additional features.

12. Limitations

1. The records are stored in shared memory and are not permanently saved to disk.
2. The system supports up to 100 student-subject records.
3. Marks must be between 0 and 100.
4. Only the most recent previous mark is stored; the system does not maintain a complete history of updates.
5. Readers check for updates periodically, approximately every 200 milliseconds.
6. The writer must be started before the readers.
7. Both readers should be stopped before restarting the writer because restarting the writer creates a fresh shared-memory session.

8. This project is intended for learning and demonstration rather than use as a production student records system.

13. Future Enhancements

The project can be improved in several ways:

1. Add student names and other details.
2. Save student records permanently using a file or database.
3. Maintain a complete history of marks updates.
4. Add search options using roll numbers.
5. Add a menu for adding, updating, and viewing student records.
6. Improve error handling and shutdown procedures.
7. Use a dedicated reader-writer synchronization mechanism to demonstrate concurrent readers more explicitly.
8. Add more reader terminals to test multiple-reader access.

14. Conclusion

The Readers–Writers Database Simulator demonstrates how shared data can be accessed and updated by separate programs. Using student marks as an example, the writer can add records and update existing marks, while two readers display the latest data in separate terminals.

POSIX shared memory allows the programs to access common records, and a process-shared mutex protects the data during reading and writing operations.

The project provides a practical introduction to C programming, Linux processes, shared memory, mutex synchronization, and concurrency in operating systems.

15. Troubleshooting

Problem: The reader cannot open the database.

Solution: Start the writer first, then start both readers.

Problem: The executable cannot run.

Solution: Run the program using ./writer or ./reader. Do not run the executable using bash.

Problem: Compilation errors occur.

Solution: Ensure that shared.h, writer.c, and reader.c are all saved in the student_marks directory. Compile the .c files using the commands provided above.

Problem: The reader does not show the update.

Solution: Confirm that the writer reports that the shared database was updated. Make sure the writer and both readers belong to the same active session. Stop both readers before restarting the writer.

Project Summary

Project Title: Readers–Writers Database Simulator — Student Marks

Programming Language: C

Operating System: Ubuntu Linux

Compiler: GCC

Main Concepts: Shared Memory, Mutex Synchronization, Concurrency, and Operating-System Processes

Number of Programs: One Writer and Two Reader Instances